The Inefficiency of Language Models in Scholarly Retrieval: An Experimental Walk-through

Shruti Singh and **Mayank Singh**
Department of Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gujarat, India
{singh_shruti, singh.mayank}@iitgn.ac.in

Abstract

Language models are increasingly becoming popular in AI-powered scientific IR systems. This paper evaluates popular scientific language models in handling (i) *short-query texts* and (ii) *textual neighbors*. Our experiments showcase the inability to retrieve relevant documents for a short-query text even under the most relaxed conditions. Additionally, we leverage textual neighbors, generated by small perturbations to the original text, to demonstrate that not all perturbations lead to close neighbors in the embedding space. Further, an exhaustive categorization yields several classes of orthographically and semantically related, partially related and completely unrelated neighbors. Retrieval performance turns out to be more influenced by the surface form rather than the semantics of the text.

1 Introduction

Representation learning methods have drastically evolved large scientific volume exploration strategies. The popular applications include summarization, construction of mentor-mentee network (Ke et al., 2021), recommendation (Ostendorff et al., 2020; Cohan et al., 2020; Das et al., 2020; Hope et al., 2021), QA over scientific documents (Su et al., 2020), and verification of scientific claims (Wadden et al., 2020). The growing community's interest has led to the development of several scientific document embedding models such as OAG-BERT (Liu et al., 2021), SPECTER (Cohan et al., 2020), SciBERT (Beltagy et al., 2019), and BioBERT (Lee et al., 2020) over the past five years. OAG-BERT has been deployed in the Aminer production system. Given similar possibilities of future deployments of scientific document embeddings models in the existing scholarly systems, it is crucial to evaluate and identify limitations robustly. However, we do not find any existing work that critically analyzes the scientific language models to the best of our knowledge.

To motivate the reader, we present a simple experiment. Queries *'document vector'* and *'document vectors'* fetch no common candidates among first page results on Google Scholar and Semantic Scholar (candidates in Appendix A). This illustrates the extremely brittle nature of such systems to minor alterations in query text, leading to completely different search outcomes. To motivate further, we experiment with textual queries encoded by the popular SciBERT model. A perturbed text *'documen vector'* (relevant in AI) is closer to the Biomedical term *'Virus vector'* in the embedding space. Similar observations were found for many other queries. As scholarly search and recommendation systems are complex systems and their detailed algorithms are not publicly available, we analyze the behavior of scientific language models, which are (or will be) presumably an integral component of each of these systems. Motivated by the usage of perturbed inputs to stress test ML systems in interpretability analysis, we propose to use *'textual neighbors'* to analyze how they are represented in the embedding space of scientific LMs. Unlike previous works (Ribeiro et al., 2020; Rychalska et al., 2019), which analyze the effect of perturbations on downstream task-specific models, we focus on analyzing the embeddings which are originally inputs to such downstream models. With the explosion of perturbation techniques for various kinds of robustness and interpretability analysis, it is difficult to generalize the insights gathered from perturbation experiments. We propose a classification schema based on orthography and semantics, to organize the perturbation strategies.

The distribution of various types of textual neighbors in a training corpus is non-uniform. Specifically, the low frequency of textual neighbors results in non-optimized representations, wherein semantically similar neighbors might end up distant in the space. These non-optimal representations have a cascading effect on the downstream task-specific

Findings of the Association for Computational Linguistics: ACL 2022, pages 3153 - 3173

models. In this work, we analyze whether all textual neighbors of input X are also X's neighbors in the embedding space. Further, we also study if their presence in the embedding space can negatively impact downstream applications dependent on similarity-based document retrieval. Our main contributions are:

1. We introduce (in Section 3) five textual neighbor categories based on orthography and semantics. We further construct a non-exhaustive list of thirty-two textual neighbor types and organize them into these five categories to analyze the behavior of scientific LMs on manipulated text.
2. We conduct (in Section 5) robust experiments to showcase the limitations of scientific LMs under a short-text query retrieval scheme.
3. We analyze (in Section 6) embeddings of textual neighbors and their placement in the embedding space of three scientific LMs, namely SciBERT, SPECTER, and OAG-BERT. Our experiments highlight the capability and limitations of different models in representing different categories of textual neighbors.

2 Related Works

Several works utilize Textual Neighbors to interpret decisions of classifiers (Ribeiro et al., 2016; Gardner et al., 2020), test linguistic capabilities of NLP models (Ribeiro et al., 2020), measure progress in language generation (Gehrmann et al., 2021), and generate semantically equivalent adversarial examples (Ribeiro et al., 2018). Similar to these works, we use textual neighbors of scientific papers to analyze the behavior of scientific LMs. MacAvaney et al. (2020) analyze the behavior of neural IR models by proposing test strategies: constructing test samples by controlling specific measures (e.g., term frequency, document length), and by manipulating text (e.g., removing stops, shuffling words). This is closest to our work, as we also employ text manipulation to analyze the behavior of scientific language models using a simple Alternative-Self Retrieval scheme (Section 4). However, our focus is not evaluation of retrieval-augmented models and we only use a relaxed document retrieval pipeline in our evaluation to analyze the behavior of scientific LMs trained on diverse domains, in encoding scientific documents. We organize the textual neighbors into categories which capture different capabilities of LMs. We also show that it is crucial to evaluate models on dissimilar texts rather than just semantically similar textual neighbors. Ours is a first work in analyzing the properties of scientific LMs for different inputs and can be utilized by future works to design and evaluate future scientific LMs. Due to space limitations, we present the detailed discussion on scientific LMs in Appendix C.

3 Short Queries and Textual Neighbors

In this paper, we experiment with short queries to fetch relevant scientific documents. The term *'short'* signifies a query length comparable to the length of research titles. The candidates are constructed from either title (T) or title and abstract (T+A) text. We, further, make small alterations to the candidate text to construct *'textual neighbors'*. The textual neighbors can be syntactically, semantically, or structurally similar to the candidate text. Unlike previous works that explore textual neighbors to analyze and stress test complex models (Q&A, Sentiment, NLI, NER (Rychalska et al., 2019)), we experiment directly with representation learning models and analyze the placement of textual neighbors in their embedding space. While semantically similar neighbors are frequently used in previous works (Ribeiro et al., 2018); we also explore *semantically dissimilar* textual neighbors to analyze scientific language models. While an LM is expected to represent semantically similar texts with high similarity, some orthographically similar but semantically dissimilar texts can have highly similar embeddings, which is undesirable behavior. Note that we restrict the current query set to titles for two main reasons: (i) most real-world search queries are shorter in length, and (ii) flat keyword-based search lacks intent and can lead to erroneous conclusions.

Textual neighbors have a similar word form or similar meaning. We observe that Textual neighbors possess the following properties based on two aspects: (i) Orthography (surface-level information content of text in terms of characters, words, and sentences), and (ii) Semantics. These two aspects are integral for a pair of texts to be textual neighbors. The properties of these aspects are:

1. **Orthography**: Orthographic-neighbors are generated by making small surface-level transformations to the input. Based on the textual content, neighbors can be generated by:
   (a) ***Lossy Perturbation (LO)***: 'Lossy' behavior indicates that the information

O-S	Neighbor Code	Form
LL-PS	T_ARot	Title → Preserve Abs → Rotate
LL-PS	T_AShuff	Title → Preserve Abs → Shuffle
LL-PS	T_ASortAsc	Title → Preserve Abs → Sort Ascending
LL-PS	T_ASortDesc	Title → Preserve Abs → Sort Descending
LO-PS	T_ADelRand	Title → Preserve Abs → Random word deletion 30%
LO-PS	T_ADelADJ	Title → Preserve Abs → Delete all ADJs
LO-DS	T_ADelNN	Title → Preserve Abs → Delete all NNs
LO-PS	T_ADelVB	Title → Preserve Abs → Delete all Verbs
LO-PS	T_ADelADV	Title → Preserve Abs → Delete all ADVs
LO-PS	T_ADelPR	Title → Preserve Abs → Delete all PRs
LO-HS	T_ADelDT	Title → Preserve Abs → Delete all DTs
LO-PS	T_ADelNum	Title → Preserve Abs → Delete all Numbers
LO-DS	T_ADelNNPH	Title → Preserve Abs → Delete all NN Phrases
LO-PS	T_ADelTopNNPH	Title → Preserve Abs → Delete top 50% NPs
LO-HS	TDelADJ_A	Title → Delete all ADJs Abs → Preserve
LO-HS	TDelNN_A	Title → Delete all NNs Abs → Preserve
LO-HS	TDelVB_A	Title → Delete all VBs Abs → Preserve
LO-HS	TDelDT_A	Title → Delete all DTs Abs → Preserve
LO-DS	TDelNN	Title → Delete all NNs Abs → Delete
LO-PS	T_ADelQ1	Title → Preserve Abs → Delete quantile 1
LO-PS	T_ADelQ2	Title → Preserve Abs → Delete quantile 2
LO-PS	T_ADelQ3	Title → Preserve Abs → Delete quantile 3
LL-HS	TNNU_A	Title → Uppercase NNs Abs → Preserve
LL-HS	TNonNNU_A	Title → Uppercase non NNs Abs → Preserve
LL-HS	T_ANNU	Title → Preserve Abs → Uppercase NNs
LL-HS	T_ANonNNU	Title → Preserve Abs → Uppercase non NNs
LO-PS	T_A_DelNNChar	Title → Delete chars from NNs Abs → Delete chars from NNs
LO-HS	TRepNNT_A	Title → Add a NN from title Abs → Preserve
LO-HS	TRepNNA_A	Title → Add a NN from abs Abs → Preserve
LO-DS	T_ADelNonNNs	Title → Preserve Abs → Delete all non NNs
LO-DS	T_ARepADJ	Title → Preserve Abs → Replace ADJs with antonyms
LL-HS	T_A_WS	Randomly replace 50% whitespace chars with 2-5 whitespace chars in the title & abstract

Table 1: Neighbor code is in format Txx_Ayy_zz, where xx and yy denote the perturbation to the paper title (T) and the abstract (A) respectively. zz denotes perturbation to both T and A. Missing T or A denotes that the corresponding input field is deleted completely.

from the original text is lost, either due to addition or deletion of textual content. E.g., random deletion of 2–3 characters from a word.

(b) ***Lossless Perturbation (LL)***: 'Lossless' behavior indicates that the original information content is unaltered. E.g., shuffling sentences of a paragraph.

2. **Semantics**: Textual neighbors can either be semantically similar or not. While semantic categories are subjective, we try to define them objectively:

   (a) ***Dissimilar (DS)***: Semantically dissimilar neighbors are generated by deletion of nouns (NNs) or noun phrases (NPs), or by changing the directionality of text such as replacing adjectives (ADJs) with antonyms.

   (b) ***Partially Similar (PS)***: Partially similar neighbors are constructed by sentence level modifications such as sentences scrambling (while preserving the word order in each sentence), word deletion of non-NNs (or NPs), or character deletion in words (including NNs) such that the textual neighbor preserves at least 70% of the words in the original text.

   (c) ***Highly Similar (HS)***: Highly similar neighbors are constructed by (i) non-word-semantic changes such as changing the case of words or altering the whitespace characters, (ii) word deletions or additions such that the textual neighbor preserves at least 90% of the words in the original text. Note that deletion of NNs only from the title while preserving the abstract will qualify to be the *HS* category.

Since SPECTER and OAG-BERT utilize paper titles and abstracts to learn embeddings, we generate textual neighbors by altering texts of these two input fields. We present 32 textual neighbor types in Table 1. Each of these 32 neighbor types is categorized into one of the five categories: LO-HS, LO-PS, LO-DS, LL-HS, and LL-PS. We exclude the LL-DS category (e.g., scrambling all words in the text) as it is infrequent and less probable to occur in a real-world setting. Examples of the textual neighbors are presented in Appendix B (Table 7).

4 Experiment Design

The Alternative-Self Retrieval: We propose an embarrassingly simple binary retrieval scheme

Dataset	Domain	SubDomain	#Papers
ACL-ANTH	CS	NLP	35,482
ICLR	CS	Deep Learning	5,032
Arxiv-CS-SY	CS	Systems and Control	10,000
Arxiv-MATH	MT	Algebraic Topology	10,000
Arxiv-HEP	HEP	HEP Theory	10,000
Arxiv-QBIO	QB	Neurons and Cognition	6,903
Arxiv-ECON	ECO	Econometrics, General & Theoretical Economics	4,542

Table 2: Dataset Statistics

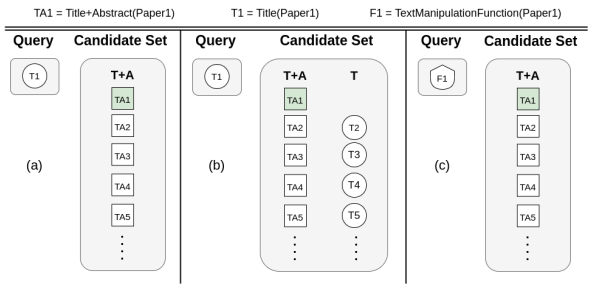


Figure 1: Alternative-Self Retrieval schemes for (a) Sec. 5 Task I, (b) Sec. 5 Task II, and (c) Sec. 6.2. Green represents the relevant candidate document for the query. The query is a subset of the relevant candidate document in schemes (a) and (b), and a textual neighbor of the relevant candidate in scheme (c).

which contains only one relevant document in the candidate set. Alternative-Self Retrieval refers to the characteristic that the query is an altered version of the relevant candidate document. E.g., the candidate documents are the embeddings of paper title and abstract (henceforth T+A) and the query is embedding of title. We present a schematic of three Alternative-Self Retrieval schemes in Figure 1. The retrieval is simple and we measure performance with accommodating metrics discussed in this section further. Our purpose is to analyze scientific LM embeddings under the most relaxed conditions.

The Datasets: We evaluate the scientific LMs on seven datasets (statistics in Table 2) to understand their effectiveness in encoding documents from diverse research fields. Each dataset contains the titles and abstracts of papers. We curate the ACL Anthology dataset[1] and the ICLR dataset from OpenReview[2]. To control the size of the ACL Anthology dataset, we exclude papers from workshops and non-ACL venues. We also curate five datasets from arXiv for the domains Mathematics (MT), High Energy Physics (HEP), Quantitative Biology (QB), Economics (ECO), and Computer Science (CS). We make available our code and dataset for public access[3].

The Notations: $\mathcal{D}$ is the set of seven datasets described in Table 2. $\mathcal{X}$ is the set of original input texts to the scientific LMs consisting of the paper title (T) and the abstract (A). For d $\in \mathcal{D}$, $\mathcal{X}$ = $\{x_j : x_j = \text{(T+A)}(p)$, where (T+A)$(p)$ = concat(title(p), abstract(p), $\forall$ paper $p \in$ d$\}$. f represents the type of textual neighbor (represented by the neighbor code presented in Table 1). $\mathcal{Q}$ and $\mathcal{R}$ are the query and candidate set for the IR task.

Evaluation Metrics: We report performance scores on the following retrieval metrics:

Mean Reciprocal Rank (MRR): All our tasks use binary relevance of documents to compute MRR.

T100: It represents the percentage of queries which retrieve the one and only relevant document among the top-100 documents.

NNk_Ret: % of queries in textual neighbor category whose k nearest neighbors (k-NN) retrieve the original document.

AOP-10: Average overlap percentage among 10-NN of x and y, where x = T+A(x_j) and y = $f(x_j)$. f is a text manipulation function, represented by the textual neighbor codes presented in Table 1.

5 Analysing Embeddings for Scientific Document Titles and Abstracts

In this section, we experiment with the inputs to the scientific language models. Due to free availability and ease of parsing paper title and abstract, majority scientific LMs learn embeddings from the title and the abstract of the paper. However, multiple downstream applications such as document search involve short queries (often keywords). We present two Alternative-Self retrieval experiments to compare the embeddings of paper title (T) with the embeddings of paper titles and abstract (T+A). In both experiments, = 1000 queries for each dataset.

Task I: Querying titles against original candidate documents In this Alternative-Self Retrieval setup, given a query q constructed only from the paper title, the system recommends relevant candidate document embeddings constructed from title and abstract both (T+A). This setting is similar to querying in a scientific literature search engine as the search queries are usually short. The

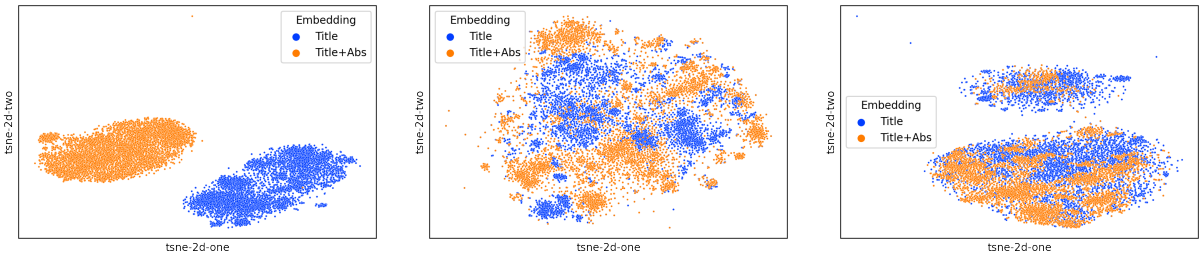
[1]https://aclanthology.org/

[2]https://openreview.net/group?id=ICLR.cc

[3]https://github.com/shruti-singh/scilm_exp

	Task I: $\mathcal{R} = \mathcal{X}$						Task II: $\mathcal{R} = \mathcal{X} \cup \mathrm{T}(\mathcal{X})$					
	SciBERT		SPECTER		OAG-BERT		SciBERT		SPECTER		OAG-BERT	
	MRR	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*
Arxiv-MATH	0.007	5.7	**0.596**	**90.5**	0.143	25.1	0	0	**0.276**	**64.2**	0.133	31.2
Arxiv-HEP	0.006	5.8	**0.693**	**92.4**	0.182	29.7	0	0	**0.354**	**75.7**	0.193	39.0
Arxiv-QBIO	0.008	6.6	**0.789**	**97.5**	0.187	33.0	0	0.2	**0.507**	**90.6**	0.194	36.1
Arxiv-ECON	0.009	10.4	**0.783**	**96.2**	0.177	32.5	0	0.1	**0.519**	**87.5**	0.176	37.2
Arxiv-CS_SY	0.011	5.0	**0.859**	**99.2**	0.186	32.1	0	0.2	**0.553**	**92.9**	0.163	34.0
ICLR	0.004	5.5	**0.586**	**91.5**	0.140	29.8	0	0	**0.221**	**66.5**	0.128	33.0
ACL-ANTH	0.002	2.3	**0.739**	**94.3**	0.138	26.5	0	0	**0.315**	**77.0**	0.101	27.3

Table 3: For both Task I and Task II, SPECTER consistently performs the best on all datasets. For Task II, drop in MRR and T100 scores for SPECTER is significant in comparison to Task I.



(a) SciBERT (b) SPECTER (c) OAG-BERT

Figure 2: t-SNE plots for T and T+A embeddings for the ICLR dataset. Completely non-overlapping embeddings for T and T+A by SciBERT model highlight differences in encoding texts of varying lengths.

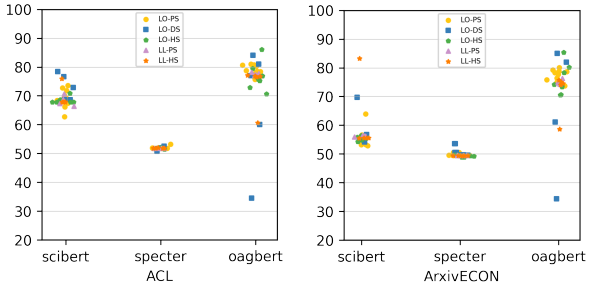


Figure 3: Percentage of pair of documents for each Textual Neighbor whose similarity is greater than the average similarity. OAG-BERT has high inter similarity (> 50%), i.e. more than 50% document pairs have cosine similarity greater than average similarity.

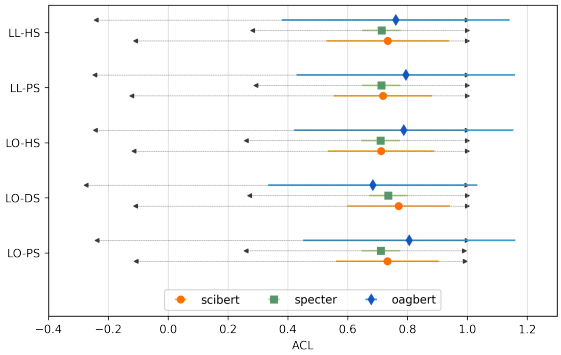


Figure 4: Inter Similarity of Textual Neighbor Vectors. Bold lines represent the μ and the σ of pairwise similarities. Arrow heads represent min and max values. Pairwise similarities are spread out in a broad range for OAG-BERT, suggesting vectors for textual neighbors are more spread out in the vector space.

motivation behind this experiment is to analyse the similarity among the T and the T+A embedding of a paper (Figure 1(a)). The experiment details are:
Query: $\mathcal{Q} = \{q_j : x_j \in \mathcal{X}, f = \mathrm{T}, q_j = f(x_j)\}$
Candidate Documents: $\mathcal{R} = \{x_k : x_k \in \mathcal{X}\}$
Task: For $q_j \in \mathcal{Q}$, rank the candidates based on cosine similarity.
Evaluation: MRR and T100. For each q_j, there is only one relevant document in the candidate set which is the corresponding T+A embedding.
We present the results for various models on different domains in Table 3. The results suggest that SciBERT performs poorly for all domains. OAG-BERT on an average ranks the original document in the range 5-7th position. However, we also observe that even in the best case, only 33% queries retrieve the original document in the top-100 retrieved candidates. SPECTER, on the other hand performs consistently better than both SciBERT and OAG-BERT. The MRR score suggests that on average, the original document is ranked in the top-2 documents, also has a good T100 score across all domains. However, for around 10% of the queries in the Arxiv-MATH, Arxiv-HEP, and ICLR datasets, SPECTER does

not rank the original relevant document in the top-100 retrieved candidates.

Task II: Introducing all Titles in the Candidate Set To increase the complexity of the previous task, we add all the title embeddings (T) in the candidate set (Figure 1(b)). We test if the T embeddings are more similar to other titles, or to their corresponding T+A embeddings.

Query: $\mathcal{Q} = \{q_j: x_j \in \mathcal{X}, f = \text{T}, q_j = f(x_j)\}$

Candidate Documents: For query q_j, the candidate set $\mathcal{R}_j$ is defined as,

$\mathcal{R}_j = \{f(x_i) : x_i \in \mathcal{X}, f = \text{T}, i \neq j\} \bigcup \{x_k: x_k \in \mathcal{X}\}$

The task and evaluation metrics are same as Task I. Extremely poor values for SciBERT (Table 3) lead us to examine the vector space of embeddings presented in Figure 2 (t-SNE plots for T and T+A embeddings), revealing that T and T+A embeddings form two non overlapping clusters. Even though the title text is a subset of T+A, SciBERT embeddings are significantly different, suggesting that input length influences SciBERT. This highlights the issue in retrieval for varying length query and candidates. We present the t-SNE plots for other datasets in the Appendix D.

SPECTER still performs the best, but a significant drop in MRR and T100 suggests that both T and T+A embeddings tightly cluster together in partially overlapping small groups (can be verified from Figure 2). However, comparable T100 for OAG-BERT to the previous experiment suggests that the model does not falter when the input text length is small. Ideally, we expect T and T+A embeddings to overlap, indicating that the embeddings of the same paper are closer. The pretraining of these models could be the reason for such distribution of T and T+A embeddings. SciBERT is trained on sentences from full text of research papers, leading to different representations for short (T) and longer (T+A) texts. As SPECTER and OAG-BERT are trained on title and abstract fields both, such non-overlapping behavior is not observed.

6 Analysing Scientific LMs with Textual Neighbors

In the previous section, we experimented with different input fields (T vs T+A). In this section, we experiment with the 32 textual neighbors classes (which alter different input fields: T, A, or T+A). We present our results for the following experiments for the five broad categories: LL-HS, LL-PS, LL-DS, LO-HS, and LO-DS. Due to space constraints, we present plots for selective datasets for each of the experiment in the paper. The rest of the plots are presented in Appendix E.

6.1 Distribution of Textual Neighbors in the Embedding Space

We measure how textual neighbor embeddings are distributed in the embedding space in each dataset when encoded by the SciBERT, SPECTER, and OAG-BERT model. For each textual neighbor class listed in Table 1, we compute pairwise similarities among all input pairs. A plot of the similarity values for different textual neighbor categories is presented in Figure 4 (additional plots in Appendix E.2). It can be observed that the pairwise similarities among documents are spread out in a significantly broader range for OAG-BERT than SciBERT and SPECTER on all datasets. The average similarity for all datasets by all models is above 0.5. We do not observe any significant difference in the average similarity for different textual neighbor classes. Interestingly, for the SPECTER model, the minimum similarity is greater than zero for all datasets across all neighbor categories. Document pair similarity via OAG-BERT embeddings have a low average similarity for the LO-DS category.

We present the percentage of pair of documents for each Textual Neighbor whose similarity is greater than the average similarity in Figure 3 (additional plots in Appendix E.2). OAG-BERT shows high inter similarity (greater than 50%) for majority of textual neighbors suggesting that more than 50% document pairs have a cosine similarity greater than average similarity. For SPECTER vectors, all types of textual neighbors have around 50% documents pairs having a similarity greater than mean similarity. However, extremely high values of percentage of document pairs having similarity greater than average similarity for the SciBERT model on the ACL dataset, and the OAG-BERT on almost all dataset suggests that majority of the documents in the embedding space are represented compactly for all textual neighbor categories.

6.2 Similarity of Textual Neighbors with Original Documents

Let $\mathcal{F} = \{f^1, f^2, ..., f^n\}$ be the set of textual neighbor functions described in Table 1. We query different types of textual neighbor classes against the documents embeddings (T+A). We compute the

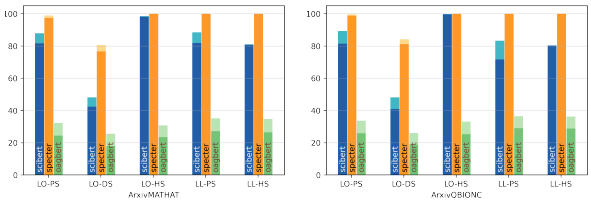


Figure 5: The bottom and the stacked bars represent NN1_Ret and NN10_Ret respectively. Results suggest that SciBERT embeddings for textual neighbors of scientific text are the most optimal.

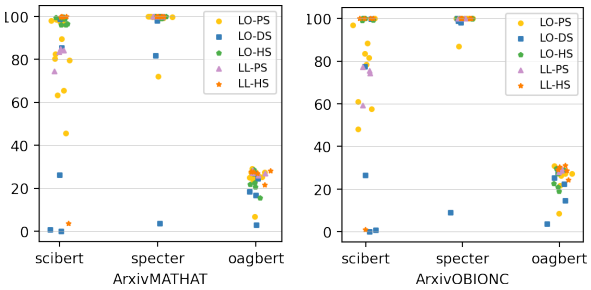


Figure 6: Distribution of NN1_Ret for each textual neighbor category. SciBERT embeddings preserve the hierarchy of NN1_Ret, i.e. PS categories (LO-PS and LL-PS) have lower values than HS categories (LO-HS and LL-HS).

percentage of queries that successfully rank the original document in the top-1 and top-10 ranked list. We expect *HS* and *PS* categories to rank the original document higher in the rank list, and *DS* to rank it lower. If any of the textual neighbor classes or categories don't show the expected behavior, it can be asserted that the LM is brittle in representing the specific type of textual neighbor.

Query: $\mathcal{Q} = \bigcup_{f \in \mathcal{F}} \mathcal{Q}_f = \bigcup_{f \in \mathcal{F}} \{q_j : x_j \in \mathcal{X}, q_j = f(x_j)\}$

Candidate Documents: $\mathcal{R} = \{x_k : x_k \in \mathcal{X}\}$

Task: For $q_j \in \mathcal{Q}$, retrieve the most similar documents based on cosine similarity.

Evaluation: NN1_Ret and NN1_10. There is only one relevant document in the candidate set for each q_j, which is the corresponding T+A embeddings.

We present the results in Figure 5. SciBERT and OAG-BERT for the LO-DS category show less than 50% NN10_Ret, which is desirable as LO-DS neighbors are semantically dissimilar, and hence should not be neighbors in the embedding space. SciBERT shows improvement via NN10_Ret over NN1_Ret for *PS* categories. High NN1_Ret for *HS* categories indicates SciBERT successfully encodes highly similar texts closer than partially similar texts. OAG-BERT performs poorly for both metrics, indicating that it doesn't encode textual

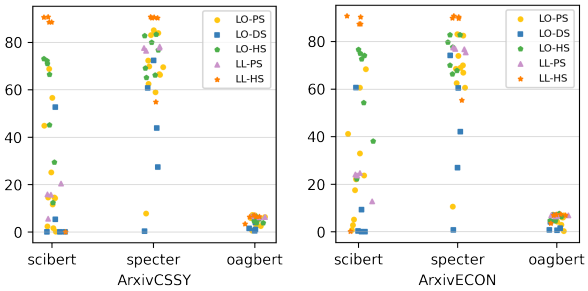


Figure 7: AOP-10 distribution of all categories of textual neighbors. SciBERT performs poorly for LL-PS (consists of neighbors that scramble abstract sentences). If we ignore LO-DS category, SPECTER embeddings perform decently overall.

neighbors optimally. SPECTER embeddings perform poorly on the LO-DS category. They however achieve the maximum values showing no difference between LO vs LL, or HS vs PS categories.

To analyse the high values for SPECTER, we present the individual NN1_Ret for each of the 32 textual neighbor classes in Figure 6 and observe only two classes 'TDelNN' and 'T_A_DelNNChar' which lead to less than 90% NN1_Ret. Unlike SPECTER and OAG-BERT, SciBERT preserve the hierarchy, with *Highly Similar* classes ranked higher than *Partially Similar* classes. An interesting case with SciBERT embeddings is that the T_A_WS neighbor class belonging to the LL-HS category, has a low NN1_Ret value across all datasets, suggesting that the SciBERT model is extremely brittle to white space character perturbations (because of the constraint on sequence length). Another breaking point for the SciBERT is the textual neighbor class T_ARepADJ (replacing adjectives with antonyms) of LO-DS category, which shows high values (around 80%) for NN1_Ret which is undesirable. We observe that among the three specific LO-PS categories 'T_ADelQ1', 'T_ADelQ2', and 'T_ADelQ3', SciBERT performs worst for the 'T_ADelQ3', indicating that the last quantile of the abstract contains relevant information encoded by SciBERT. OAG-BERT shows a reverse trend to SPECTER by achieving low values for all neighbors classes indicating brittleness to text manipulation.

6.3 Overlap amongst Nearest Neighbors

We compute the overlap amongst the nearest neighbors of each textual neighbor class and the original document embeddings. We randomly sample a query set of 2000 queries for each textual neighbor

TN Cat	LO-HS			LO-PS			LO-DS			LL-HS			LL-PS		
Model	**SB**	**SP**	**OB**	**SB**	**SP**	**OB**	**SB**	**SP**	**OB**	**SB**	**SP**	**OB**	**SB**	**SP**	**OB**
ACL-ANTH	51.8	**68.9**	4.4	20.5	**61.8**	4.3	12.0	35.6	**2.2**	73.0	**81.2**	5.1	13.4	**72.0**	5.2
ICLR	52.2	**74.6**	4.7	18.6	**61.8**	4.7	10.9	33.2	**2.3**	71.2	**82.2**	5.5	10.8	**75.0**	6.0
Arxiv-CS_SY	52.8	**74.8**	5.2	23.2	**66.0**	5.1	11.6	40.9	**2.6**	71.7	**83.4**	6.0	14.5	**77.6**	6.6
Arxiv-MATH	52.2	**67.4**	5.3	23.0	**62.1**	5.3	14.5	34.5	**3.2**	71.1	**80.5**	6.1	31.3	**75.4**	6.6
Arxiv-ECON	59.0	**75.3**	5.8	27.0	**66.4**	5.6	14.1	40.9	**2.8**	71.2	**83.2**	6.4	21.4	**76.8**	6.8
Arxiv-QBIO	55.2	**74.1**	5.4	23.5	**65.1**	4.9	12.1	38.7	**2.6**	71.0	**82.9**	5.9	17.1	**76.4**	6.3
Arxiv-HEP	53.1	**66.2**	5.9	22.9	**61.0**	5.9	13.5	34.2	**3.6**	71.0	**80.0**	6.8	25.4	**73.4**	7.2

Table 4: AOP-10 values for different categories. The best results for LO-DS category are from OAG-BERT (OB), however that is because the model performs poorly on all categories of textual neighbors. Similarly, best results for the rest four categories are from SPECTER (SP), following which it also has a high overlap percentage for the LO-DS category. SciBERT (SB) embeddings perform the best for HS and DS but falter on PS semantic categories.

class (and their corresponding T+A embedding) and compute nearest neighbor (NN) overlap for these. In this task, we are interested in evaluating if NN-based retrieval retrieves the same documents for a textual neighbor class and T+A embedding.
Query: $\mathcal{Q} = \mathcal{Q}_f \cup \mathcal{Q}_{T+A}$
$\mathcal{Q}_f = \{q_j\colon x_j \in \mathcal{X}_{2000}, q_j = f(x_j)\}$
$\mathcal{Q}_{T+A} = \{q_j\colon x_j \in \mathcal{X}_{2000}, q_j = T + A(x_j)\}$
Candidate Documents: $\mathcal{R} = \{q_j\colon x_j \in \mathcal{X}, q_j{=}f(x_j)\} \cup \{q_j\colon x_j \in \mathcal{X}, f = T + A, q_j{=}f(x_j)\}$
Task: For each pair of $q_j, q_k \in \mathcal{Q}$, such that $q_j \in \mathcal{Q}_f$ and $q_k \in \mathcal{Q}_{T_A}$, compute the overlap among ten nearest neighbors (NN-10) of q_j and q_k.
Evaluation: AOP-10.
We present the results arranged by Textual Neighbor categories in Table 4. The individual AOP-10 distribution of all categories of textual neighbors is presented in Figure 7. While overall results look good for SPECTER, it should be noted that SPECTER performs poorly for embedding LO-DS textual neighbors indicating that it has a shallow understanding of semantics. AOP-10 values for SPECTER show a positive trend: LL-HS $>$ LO-HS and LL-HS $>$ LL-PS. SciBERT performs decently for the *HS* category, but its performance drops for the *PS* categories. OAG-BERT has the lowest AOP-10 for all textual neighbor categories. When put in perspective against the previous NN1_Ret and NN10_Ret, we believe that SciBERT performs decently in encoding the HS and PS neighbors closer to the original T+A embedding. However, low value of AOP-10 for SciBERT for PS neighbors reflects that while the PS neighbors are closer to the original document in comparison to others, the original document has other nearest neighbors than the PS neighbor.

We present a matrix to summarize the capability of the models in Table 5 in encoding the five textual neighbor categories. The five categories

	LL-HS	LO-HS	LL-PS	LO-PS	LO-DS
Threshold	> 75	> 70	$50 <$ AOP-20 < 70		< 20
SciBERT	**75.04**	59.62	22.6	26.28	14.35
SPECTER	**86.77**	**77.55**	80.93	**69.03**	41.32
OAG-BERT	6.84	6.24	7.31	5.95	**3.17**

Table 5: Capability of the models in encoding textual neighbor categories optimally in the embedding space. Gray cells represent optimal representations for each model based on AOP-20.

arranged in increasing order of semantic similarity are: LL-HS $\geq$ LO-HS $>$ LL-PS $>$ LO-PS $>$ LO-DS. We use heuristic-based values to define optimality. For each of the five categories, we define AOP-20 thresholds to classify if the textual neighbor representations for the corresponding are optimal or not. It is expected that AOP-20 values for semantic categories should be in order: HS $>$ PS $>$ DS. AOP-20 values for orthographic categories should follow: LL $>$ LO.

7 Conclusion

We propose five categories of textual neighbors to organize the increasing number of textual neighbor types: LL-HS, LO-HS, LL-PS, LO-PS, and LO-DS. We evaluate SciBERT, SPECTER, and OAG-BERT models on thirty-two textual neighbor classes organized into the previous five categories. We show that evaluation of language models on 'Semantically Dissimilar' texts is also important rather than just evaluation on 'Semantically Similar' texts. We show that the SciBERT model is highly sensitive to the input length. SPECTER embeddings for all types of textual neighbors are highly similar irrespective of whether the textual neighbor is semantically dissimilar or not. SPECTER embeddings show sensitivity to the presence of specific keywords. Lastly, OAG-BERT embeddings of all categories of textual neighbors are highly dissimilar to the original title and abstract (T+A) embed-

dings. We believe that our insights could be used to develop better pretraining strategies for scientific document language models and also to evaluate other language models. One example for MLM (or replaced token identification) could be utilizing a weighted-penalty based loss, i.e. partially similar tokens if predicted should be penalized less in comparison to the prediction of unrelated (or dissimilar) tokens. Additionally, these insights could also be utilised by several systems that use these scientific document language models to incorporate informed strategies in downstream systems such as recommendation systems.

References

Iz Beltagy, Kyle Lo, and Arman Cohan. 2019. SciBERT: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3615–3620, Hong Kong, China. Association for Computational Linguistics.

Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. 2020. SPECTER: document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 2270–2282. Association for Computational Linguistics.

Debasmita Das, Yatin Katyal, Janu Verma, Rajesh Kumar Ranjan, Shashank Dubey, Aakash Deep Singh, Sourojit Bhaduri, and Kushagra Agarwal. 2020. Information retrieval and extraction on covid-19 clinical articles using graph community detection and bio-bert embeddings.

Soumyajit Ganguly and Vikram Pudi. 2017. Paper2vec: Combining graph and text information for scientific paper representation. In *European conference on information retrieval*, pages 383–395. Springer.

Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco, Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, Sameer Singh, Noah A. Smith, Sanjay Subramanian, Reut Tsarfaty, Eric Wallace, Ally Zhang, and Ben Zhou. 2020. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323, Online. Association for Computational Linguistics.

Sebastian Gehrmann, Tosin Adewumi, Karmanya Aggarwal, Pawan Sasanka Ammanamanchi, Anuoluwapo Aremu, Antoine Bosselut, Khyathi Raghavi Chandu, Miruna-Adriana Clinciu, Dipanjan Das, Kaustubh Dhole, Wanyu Du, Esin Durmus, Ondřej Dušek, Chris Chinenye Emezue, Varun Gangal, Cristina Garbacea, Tatsunori Hashimoto, Yufang Hou, Yacine Jernite, Harsh Jhamtani, Yangfeng Ji, Shailza Jolly, Mihir Kale, Dhruv Kumar, Faisal Ladhak, Aman Madaan, Mounica Maddela, Khyati Mahajan, Saad Mahamood, Bodhisattwa Prasad Majumder, Pedro Henrique Martins, Angelina McMillan-Major, Simon Mille, Emiel van Miltenburg, Moin Nadeem, Shashi Narayan, Vitaly Nikolaev, Andre Niyongabo Rubungo, Salomey Osei, Ankur Parikh, Laura Perez-Beltrachini, Niranjan Ramesh Rao, Vikas Raunak, Juan Diego Rodriguez, Sashank Santhanam, João Sedoc, Thibault Sellam, Samira Shaikh, Anastasia Shimorina, Marco Antonio Sobrevilla Cabezudo, Hendrik Strobelt, Nishant Subramani, Wei Xu, Diyi Yang, Akhila Yerukola, and Jiawei Zhou. 2021. The GEM benchmark: Natural language generation, its evaluation and metrics. In *Proceedings of the 1st Workshop on Natural Language Generation, Evaluation, and Metrics (GEM 2021)*, pages 96–120, Online. Association for Computational Linguistics.

Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2020. Domain-specific language model pretraining for biomedical natural language processing. *arXiv preprint arXiv:2007.15779*.

Tom Hope, Aida Amini, David Wadden, Madeleine van Zuylen, Sravanthi Parasa, Eric Horvitz, Daniel Weld, Roy Schwartz, and Hannaneh Hajishirzi. 2021. Extracting a knowledge base of mechanisms from COVID-19 papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4489–4503, Online. Association for Computational Linguistics.

Kexin Huang, Jaan Altosaar, and Rajesh Ranganath. 2019. Clinicalbert: Modeling clinical notes and predicting hospital readmission. *CoRR*, abs/1904.05342.

Qing Ke, Lizhen Liang, Ying Ding, Stephen V David, and Daniel E Acuna. 2021. A dataset of mentorship in science with semantic and demographic estimations. *arXiv preprint arXiv:2106.06487*.

Xiangjie Kong, Mengyi Mao, W. Wang, Jiaying Liu, and Bo Xu. 2021. Voprec: Vector representation learning of papers with text information and structural identity for recommendation. *IEEE Transactions on Emerging Topics in Computing*, 9:226–237.

Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196. PMLR.

Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240.

Xiao Liu, Da Yin, Xingjian Zhang, Kai Su, Kan Wu, Hongxia Yang, and Jie Tang. 2021. Oag-bert: Pre-train heterogeneous entity-augmented academic language models. *arXiv preprint arXiv:2103.02410*.

Sean MacAvaney, Sergey Feldman, Nazli Goharian, Doug Downey, and Arman Cohan. 2020. ABNIRML: Analyzing the behavior of neural IR models. *arXiv preprint arXiv:2011.00696*.

Malte Ostendorff, Terry Ruas, Till Blume, Bela Gipp, and Georg Rehm. 2020. Aspect-based document similarity for research papers. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6194–6206, Barcelona, Spain (Online). International Committee on Computational Linguistics.

Bryan Perozzi, Rami Al-Rfou, and S. Skiena. 2014. Deepwalk: online learning of social representations. *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*.

Leonardo F.R. Ribeiro, Pedro H.P. Saverese, and Daniel R. Figueiredo. 2017. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '17, page 385–394, New York, NY, USA. Association for Computing Machinery.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, pages 1135–1144.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2018. Semantically equivalent adversarial rules for debugging NLP models. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 856–865, Melbourne, Australia. Association for Computational Linguistics.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, Online. Association for Computational Linguistics.

Barbara Rychalska, Dominika Basaj, Alicja Gosiewska, and Przemysław Biecek. 2019. Models in the wild: On corruption robustness of neural nlp systems. In *International Conference on Neural Information Processing*, pages 235–247. Springer.

Hoo-Chang Shin, Yang Zhang, Evelina Bakhturina, Raul Puri, Mostofa Patwary, Mohammad Shoeybi, and Raghav Mani. 2020. BioMegatron: Larger biomedical domain language model. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4700–4706, Online. Association for Computational Linguistics.

Yuqi Si, Jingqi Wang, Hua Xu, and Kirk Roberts. 2019. Enhancing clinical concept extraction with contextual embeddings. *Journal of the American Medical Informatics Association*, 26(11):1297–1304.

Dan Su, Yan Xu, Tiezheng Yu, Farhad Bin Siddique, Elham J. Barezi, and Pascale Fung. 2020. Caire-covid: A question answering and query-focused multi-document summarization system for COVID-19 scholarly information management. In *Proceedings of the 1st Workshop on NLP for COVID-19@ EMNLP 2020, Online, December 2020*. Association for Computational Linguistics.

Han Tian and Hankz Hankui Zhuo. 2017. Paper2vec: Citation-context based document distributed representation for scholar recommendation. *CoRR*, abs/1703.06587.

William Timkey and Marten van Schijndel. 2021. All bark and no bite: Rogue dimensions in transformer language models obscure representational quality. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4527–4546, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. 2020. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, Online. Association for Computational Linguistics.

Danhao Zhu, Xinyu Dai, and Jiajun Chen. 2019. Representing anything from scholar papers. *Journal of Web Semantics*, 59.

Appendix

A Candidate Retrieval

The candidates fetched for the queries 'document vector' and 'document vectors' on Google Scholar and Semantic Scholar in Table 6. It can be observed that both queries have no common candidates on either of the search engines.

Query = document vector	**Query = document vectors**
Semantic Scholar	Semantic Scholar
Corporate value evaluation using patent document vector	Using Sparse Composite Document Vectors to Classify VBA Macros
Improving a tf-idf weighted document vector embedding	SCDV : Sparse Composite Document Vectors using soft clustering over distributional representations
Legal Document Retrieval using Document Vector Embeddings and Deep Learning	Music genre classification with word and document vectors
Document vector embeddings for bibliographic records indexing	Constructing Document Vectors Using Kernel Density Estimates
Document Vector Extension for Documents Classification	Text classification with sparse composite document vectors
Multi-document Summarization by Creating Synthetic Document Vector Based on Language Model	Words are not Equal: Graded Weighting Model for Building Composite Document Vectors
Document vector representations for feature extraction in multi-stage document ranking	A Document Descriptor using Covariance of Word Vectors
An Adaptive Topic Tracking Model Based on 3-Dimension Document Vector	Document Embedding with Paragraph Vectors
A support vector machine mixed with TF-IDF algorithm to categorize Bengali document	Document classification with distributions of word vectors
A framework for a feedback process to analyze and personalize a document vector space in a feature extraction model	Text document clustering using global term context vectors
Google Scholar	Google Scholar
Document vector representations for feature extraction in multi-stage document ranking	Document embedding with paragraph vectors
Document ranking and the vector-space model	SCDV: Sparse Composite Document Vectors using soft clustering over distributional representations
Legal document retrieval using document vector embeddings and deep learning	Text document clustering using global term context vectors
Improving a tf-idf weighted document vector embedding	Document classification with distributions of word vectors
Efficient vector representation for documents through corruption	Using sparse composite document vectors to classify vba macros
Enhancing web service clustering using Length Feature Weight Method for service description document vector space representation	Words are not equal: Graded weighting model for building composite document vectors
Document vector compression and its application in document clustering	Image-based document vectors for text retrieval
A vector space model for automatic indexing	Improving Document Vectors Representation using Semantic Links and Attributes
Hierarchical document categorization with support vector machines	Self organization of a massive document collection
Using an n-gram-based document representation with a vector processing retrieval model	Music genre classification with word and document vectors

Table 6: Candidate documents retrieved for queries ‘document vector’ and ‘document vectors’.

B Textual Neighbors

We present examples for textual neighbors in Table 7. We use NLTK for preprocessing text and constructing textual neighbors.

C Summary of Scientific LMs

We discuss some popular scientific document language models which leverage the transformer architecture.

SciBERT (Beltagy et al., 2019) is also a BERT model trained on large amounts of scientific data. It is trained on a random sample of 1.14M papers from the Semantic Scholar Corpus. The training corpus consists of 18% papers from the computer science domain and 82% from the broad biomedical domain. Full texts of the papers are used for training.

SPECTER (Cohan et al., 2020) uses citation-informed Transformers to generate general-purpose vector representations of scientific documents. Unlike traditional models, they also leverage inter-document relatedness to learn general purpose embeddings that are effective across a variety of downstream tasks without task-specific fine-tuning. SPECTER leverages citations as a signal for document-relatedness and formulate this into a triplet-loss pre-training objective. SPECTER achieves state-of-the-art results on six out of seven document-level tasks for scientific literature in the SCIDOCS (Cohan et al., 2020) benchmark suite.

OAG-BERT (Liu et al., 2021) jointly model texts (title and abstract of the paper) and heterogeneous academic entities (authors, research field, venues, and affiliations) to learn representations for a scientific document. The architecture is similar to BERT, however the authors employ multiple techniques to learn entity embeddings. To distinguish different textual and academic entities use entity type embeddings to indicate the entity type. They design an entity aware 2D-positional encoding to indicate the inter-entity and the intra-entity sequence order. It also proposes span-aware entity masking to preserve the sequential relationship between the entity's tokens.

BioBERT (Lee et al., 2020) is a BERT model pre-trained on large-scale biomedical corpora. BioBERT model is initialized with BERT cite weights and then pre-trained on PubMed abstracts and PMC full-text articles.

Succeeding the BioBERT model, several models have been trained exclusively for Biomedical texts such as ClinicalBERT (Huang et al., 2019), MIMIC-BERT (Si et al., 2019), PubMedBERT (Gu et al., 2020), and BioMegatron (Shin et al., 2020) to list a few. However, in this work, we focus on general purpose scientific language models that have been trained on scientific documents from diverse research fields.

We summarize the details of the SciBERT, SPECTER, and the OAG-BERT model in Table 8.

C.1 Non Transformer-based Models

Majority of Non Transformer based models utilise the Paragraph Vector (Le and Mikolov, 2014) technique to learn vectors for the textual content. Citation networks are utilised to learn similar embeddings for related papers. Paper2Vec (Ganguly and Pudi, 2017) learns embeddings by applying DeepWalk (Perozzi et al., 2014) on an augmented citation network of papers. Apart from connecting cited papers, the augmented network also connects k nearest neighbors (from textual embeddings generated using Paragraph Vector (Le and Mikolov, 2014)). Paper2Vec (Tian and Zhuo, 2017) learns distributed vertex embeddings from matrix factorization on the weighted context definition of nodes. Following a similar technique as Paper2Vec (Ganguly and Pudi, 2017), VOPRec (Vector Representation Learning of Papers with Text Information and Structural Identity for Recommendation) (Kong et al., 2021) learns embeddings from the text using Paragraph Vector (Le and Mikolov, 2014) and the citation network using Struc2Vec (Ribeiro et al., 2017).

Zhu et al. (2019) present a method to learn scholar paper embeddings (Represent Anything from Scholar Papers) from different scholar entities such as title, authors, publication venue, and citations. It trains the model by trying to maximize the likelihood of references of a paper. It uses an encoder-decoder framework to learn representations from title words, author names, publication venue, and publication year. The proposed method can generate representations for papers even if the references are missing as that information is already encoded in the entities during the training.

As the pretrained models or code for none of the Non Transformer-based models is publicly available, we skip their evaluation in this work.

O-S	Neighbor Code	Form	Example
LL-PS	T_ARot	T → Preserve A → Rotate	**A:** We introduce a representation for computer programs based on language models. We train deep robust embeddings using pytorch. Contextual embeddings are common in NLP.
LL-PS	T_AShuff	T → Preserve A → Shuffle	**A:** Contextual embeddings are common in NLP. We introduce a representation for computer programs based on language models. We train deep robust embeddings using pytorch.
LL-PS	T_ASortAsc	T → Preserve A → Sort Ascending	**A:** Contextual embeddings are common in NLP. We train deep robust embeddings using pytorch. We introduce a representation for computer programs based on language models.
LL-PS	T_ASortDesc	T → Preserve A → Sort Descending	**A:** We introduce a representation for computer programs based on language models. We train deep robust embeddings using pytorch. Contextual embeddings are common in NLP.
LO-PS	T_ADelRand	T → Preserve A → Random word deletion 30%	**A:** Contextual are common in . introduce a representation computer programs based on language models. We train deep robust using .
LO-PS	T_ADelADJ	T → Preserve A → Delete all ADJs	**A:** We introduce a representation for computer programs based on language models . We train embeddings using pytorch .
LO-DS	T_ADelNN	T → Preserve A → Delete all NNs	**A:** Contextual are common in . We introduce a for based on . We train deep robust using .
LO-PS	T_ADelVB	T → Preserve A → Delete all Verbs	**A:** Contextual embeddings common in NLP . We a representation for computer programs on language models . We deep robust embeddings pytorch .
LO-PS	T_ADelADV	T → Preserve A → Delete all ADVs	**A:** Contextual embeddings are common in NLP . We introduce a representation for computer programs based on language models . We train deep robust embeddings using pytorch .
LO-PS	T_ADelPR	T → Preserve A → Delete all PRs	**A:** Contextual embeddings are common in NLP . introduce a representation for computer programs based on language models . train deep robust embeddings using pytorch .
LO-HS	T_ADelDT	T → Preserve A → Delete all DTs	**A:** Contextual embeddings are common in NLP . We introduce representation for computer programs based on language models . We train deep robust embeddings using pytorch .
LO-PS	T_ADelNum	T → Preserve A → Delete all Numbers	**A:** Contextual embeddings are common in NLP . We introduce a representation for computer programs based on language models . We train deep robust embeddings using pytorch .
LO-DS	T_ADelNNPH	T → Preserve A → Delete all NN Phrases	**A:** Contextual embeddings are common in NLP. We introduce a representation for based on . We train deep using pytorch.
LO-PS	T_ADelTopNNPH	T → Preserve A → Delete top 50% NPs	**A:** Contextual embeddings are common in NLP. We introduce a representation for based on . We train deep robust embeddings using pytorch.
LO-HS	TDelADJ_A	T → Delete all ADJs A → Preserve	**T:** Source Code Embeddings from Language Models
LO-HS	TDelNN_A	T → Delete all NNs A → Preserve	**T:** from
LO-HS	TDelVB_A	T → Delete all VBs A → Preserve	**T:** Source Code Embeddings from Language Models
LO-HS	TDelDT_A	T → Delete all DTs A → Preserve	**T:** Source Code Embeddings from Language Models
LO-DS	TDelNN	T → Delete all NNs A → Delete	**T:** from
LO-PS	T_ADelQ1	T → Preserve A → Delete quantile 1	**A:** We introduce a representation for computer programs based on language models. We train deep robust embeddings using pytorch.
LO-PS	T_ADelQ2	T → Preserve A → Delete quantile 2	**A:** Contextual embeddings are common in NLP. We train deep robust embeddings using pytorch.
LO-PS	T_ADelQ3	T → Preserve A → Delete quantile 3	**A:** Contextual embeddings are common in NLP. We introduce a representation for computer programs based on language models.
LL-HS	TNNU_A	T → Uppercase NNs A → Preserve	**T:** SOURCE CODE EMBEDDINGS from LANGUAGE MODEL
LL-HS	TNonNNU_A	T → Uppercase non NNs A → Preserve	**T:** Source Code Embeddings FROM Language Models
LL-HS	T_ANNU	T → Preserve A → Uppercase NNs	**A:** Contextual EMBEDDINGS are common in NLP . We introduce a REPRESENTATION for COMPUTER PROGRAMS based on LANGUAGE MODELS . We train deep robust EMBEDDINGS using PYTORCH .

O-S	Neighbor Code	Form	Example
LL-HS	T_ANonNNU	T → Preserve A → Uppercase non NNs	**A:** CONTEXTUAL embeddings ARE COMMON IN NLP . WE INTRODUCE A representation FOR computer programs BASED ON language models . WE TRAIN DEEP ROBUST embeddings USING pytorch .
LO-PS	T_A_DelNNChar	T → Delete chars from NNs A → Delete chars from NNs	**T:** Soue Cod Emddings from Lguage dels **A:** Contextual mbedding are common in NLP . We introduce a representaio for cmuter prrams based on lngage modl . We train deep robust emeddngs using ytorh .
LO-HS	TRepNNT_A	T → Add a NN from title A → Preserve	**T:** Source Source Code Embeddings from Language Models
LO-HS	TRepNNA_A	T → Add a NN from abs A → Preserve	**T:** Source Code Embeddings from Language Models embeddings NLP representation computer
LO-DS	T_ADelNonNNs	T → Preserve A → Delete all non NNs	**A:** NLP representation computer programs language models embeddings pytorch
LO-DS	T_ARepADJ	T → Preserve A → Replace ADJs with antonyms	**A:** Contextual embeddings are individual in NLP . We introduce a representation for computer programs based on language models . We train shallow frail embeddings using pytorch .
LL-HS	T_A_WS	Randomly replace 50% whitespace chars with 2-5 whitespace chars in T & A	**TA:** Source_Code___Embeddings_from____Language_Models._Contextual_embeddings_are_common_in_NLP._We_introduce_a_representation_for _computer_programs_based_on_language_models._We_train_deep_robust_ embeddings___using____pytorch. [WS represented using _]

Table 7: Neighbor code is in format Txx_Ayy_zz, where xx and yy denote the perturbation to the paper title (T) and the abstract (A) respectively. zz denotes perturbation to both T and A. Missing T or A denotes that the corresponding input field is deleted completely.

	OAG-BERT	SPECTER	SciBERT
Model Architecture	Entity augmented BERT-base	BERT-base	BERT-base
Model Initialization	-	SciBERT	-
Loss Function	Hybrid Cross Entropy	Triplet Margin Loss	Cross Entropy
Training corpus	Open Academic Graph	Semantic Scholar Corpus	Semantic Scholar Corpus
Vocabulary	OAG-BERT Vocab	SciVocab	SciVocab
Vocabulary Size	44,000 tokens	30,000 tokens	30,000 tokens
Tokenizer	WordPiece	WordPiece	WordPiece
Text Features	Title, Abstract, Body, FoS, Authors, Venues, Affiliations	Title, Abstract	Full-text

Table 8: Comparison of different transformer based language models for scientific literature

D Analysing Embeddings for Scientific Document Titles and Abstracts

We present the t-SNE plots for the Task II in Figure 8. The embedding space contains vectors for titles and the T+A embeddings.

D.1 Normalized Embeddings

Timkey and van Schijndel (2021) indicate that cosine similarity is dominated by few rouge dimensions in embeddings. We repeat Task I with: (i) L2-normalized embeddings, and (ii) standardization procedure by Timkey and van Schijndel (2021). While L2-normalization leads to an incremental improvement, standardization leads to improvement only for SciBERT and SPECTER models. We present the results of **Task I** (Section 5) with normalized embeddings in Table 9.

E Analysing Scientific LMs with Textual Neighbors

We present the plots for each of the seven datasets for the experiments with textual neighbors in the following sections.

E.1 Distribution of Textual Neighbors in the Embedding Space

We present the plot for inter similarity of textual neighbor vectors in Figure 9, which depicts the maximum, minimum, mean and standard deviation of all pairs of documents for the five neighbor categories. Next, we present the percentage of document pairs for each of the 32 textual neighbor classes, whose similarity is greater than the average similarity for that particular class in Figure 10.

E.2 Similarity of Textual Neighbors with Original Documents

Figure 11 presents the NN1_Ret and NN10_Ret for each of the datasets for the five textual neighbor

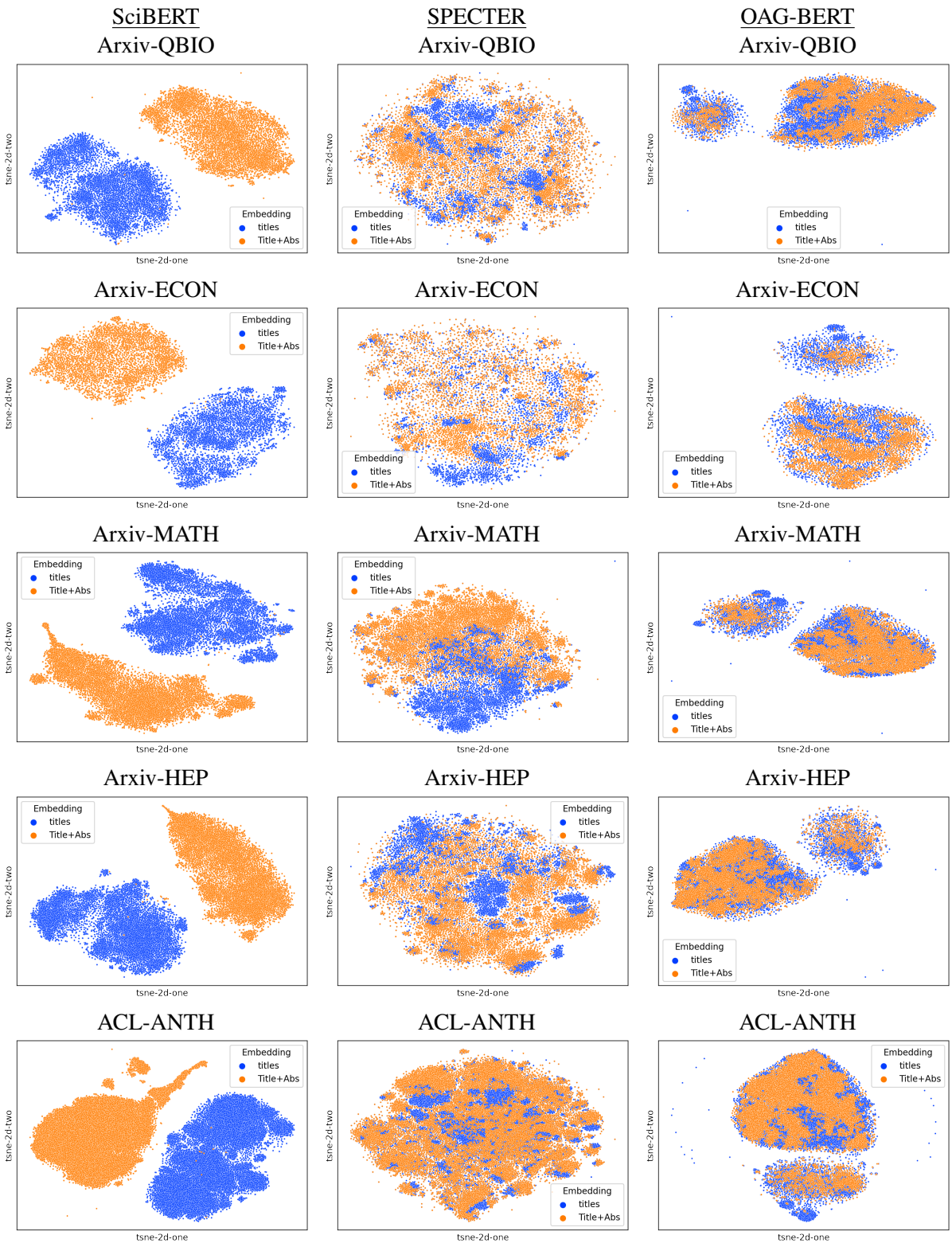


Figure 8: t-SNE plots for T and T+A embeddings for various dataset. Completely non-overlapping embeddings for T and T+A by SciBERT model highlight differences in encoding texts of varying lengths.

categories in the stacked bar format (NN10_Ret stacked onto NN1_Ret values). It can be observed that for all datasets, SPECTER achieves extremely high ($> 95\%$) NN1_Ret values for four (LO-PS, LO-HS, LL-PS, and LL-HS) out of five classes, and hence NN10_Ret does not significantly improve the retrieval. SciBERT values though less in comparison to SPECTER, show a nice trend

	L2-normalized embeddings						Standardization (Timkey and van Schijndel, 2021)					
	SciBERT		SPECTER		OAG-BERT		SciBERT		SPECTER		OAG-BERT	
	MRR	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*	*MRR*	*T100*
Arxiv-MATH	0.013	6.8	0.62	90.4	0.148	26.2	0.06	25.1	**1**	**100**	0.11	20.8
Arxiv-HEP	0.01	8.2	0.693	93.8	0.181	30.2	0.07	29.5	**1**	**100**	0.126	23.7
Arxiv-QBIO	0.007	6.8	0.795	98.0	0.182	31.0	0.144	54.5	**1**	**100**	0.127	24.1
Arxiv-ECON	0.01	11.4	0.772	95.9	0.196	34.8	0.142	49.8	**1**	**100**	0.144	26.6
Arxiv-CS_SY	0.007	5.9	0.856	99.4	0.183	31.3	0.116	47.9	**1**	**100**	0.132	23.9
ICLR	0.007	5.4	0.592	91.8	0.145	29.9	0.09	41.0	**1**	**100**	0.124	25.1
ACL-ANTH	0.002	3.1	0.74	95.1	0.123	25.2	0.034	18.2	**0.99**	**100**	0.096	18.2

Table 9: L2 normalization leads to an incremental improvement in performance. Standardization leads to improvement for SciBERT and SPECTER models, but the same effect is not observed for OAG-BERT embeddings.

where NN10_Ret does not improve the retrieval significantly for *HS* categories (LO-HS and LL-HS), however it does improve retrieval recall for *PS* categories (LO-PS and LL-PS). OAG-BERT performs poorly with all categories achieving NN10_Ret values less than 40%. We present in Figure 12, the NN1_Ret for each of the 32 textual neighbor classes. A close inspection reveals an interesting case for SciBERT, which has extremely low NN1_Ret values ($< 10\%$) for one of the LL-HS category, T_A_WS whih replaces 50% whitespace characters randomly with 2-5 whitespaces.

E.3 Overlap amongst Nearest Neighbors

AOP-10 is the average overlap percentage among the 10-NN (10 nearest neighbors) of the original document embeddings (T+A) and the textual neighbor embeddings. AOP-10 distribution of the 32 textual neighbor classes is presented in Figure 13. Low overlap percentage for OAG-BERT suggests that the model falters when presented with textual neighbors and does not represent textual neighbors in the neighborhood of the original document embeddings in the embedding space.

Figure 9: Inter Similarity of Textual Neighbor Vectors. The pairwise similarities among documents are spread out in a significantly broader range for OAG-BERT than SciBERT and SPECTER, suggesting that OAG-BERT vectors for different textual neighbors are more spread out in the vector space.

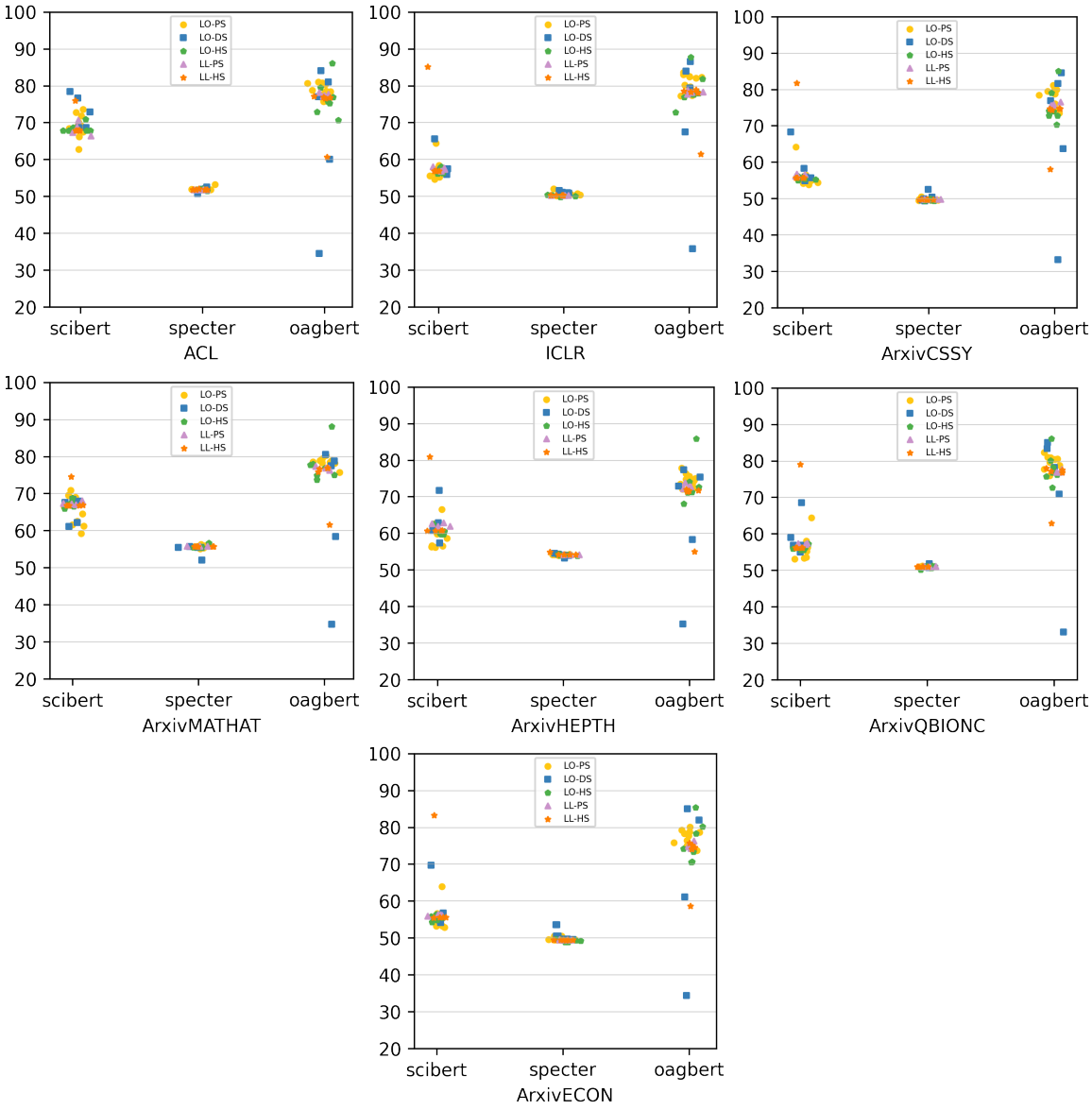


Figure 10: Percentage of pair of documents for each Textual Neighbor whose similarity is greater than the average similarity. OAG-BERT shows high inter similarity (greater than 50%) among all textual neighbors suggesting that more than 50% document pairs have a cosine similarity greater than average similarity.

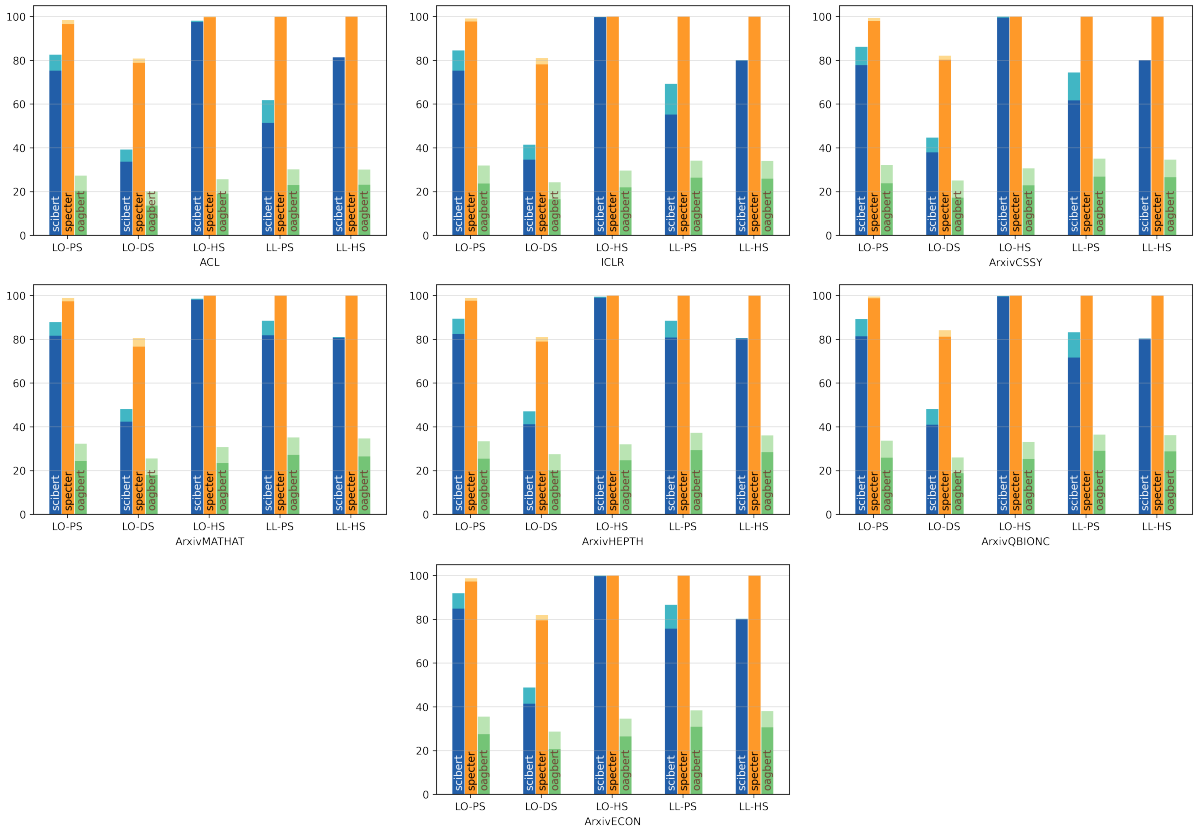


Figure 11: Stacked bars representing the percentage of documents of each textual neighbor category which retrieve the original document in the top-1 nearest neighbor and the top-10 nearest neighbor respectively. Results suggest that SciBERT embeddings for textual neighbors of scientific text are the most optimal.

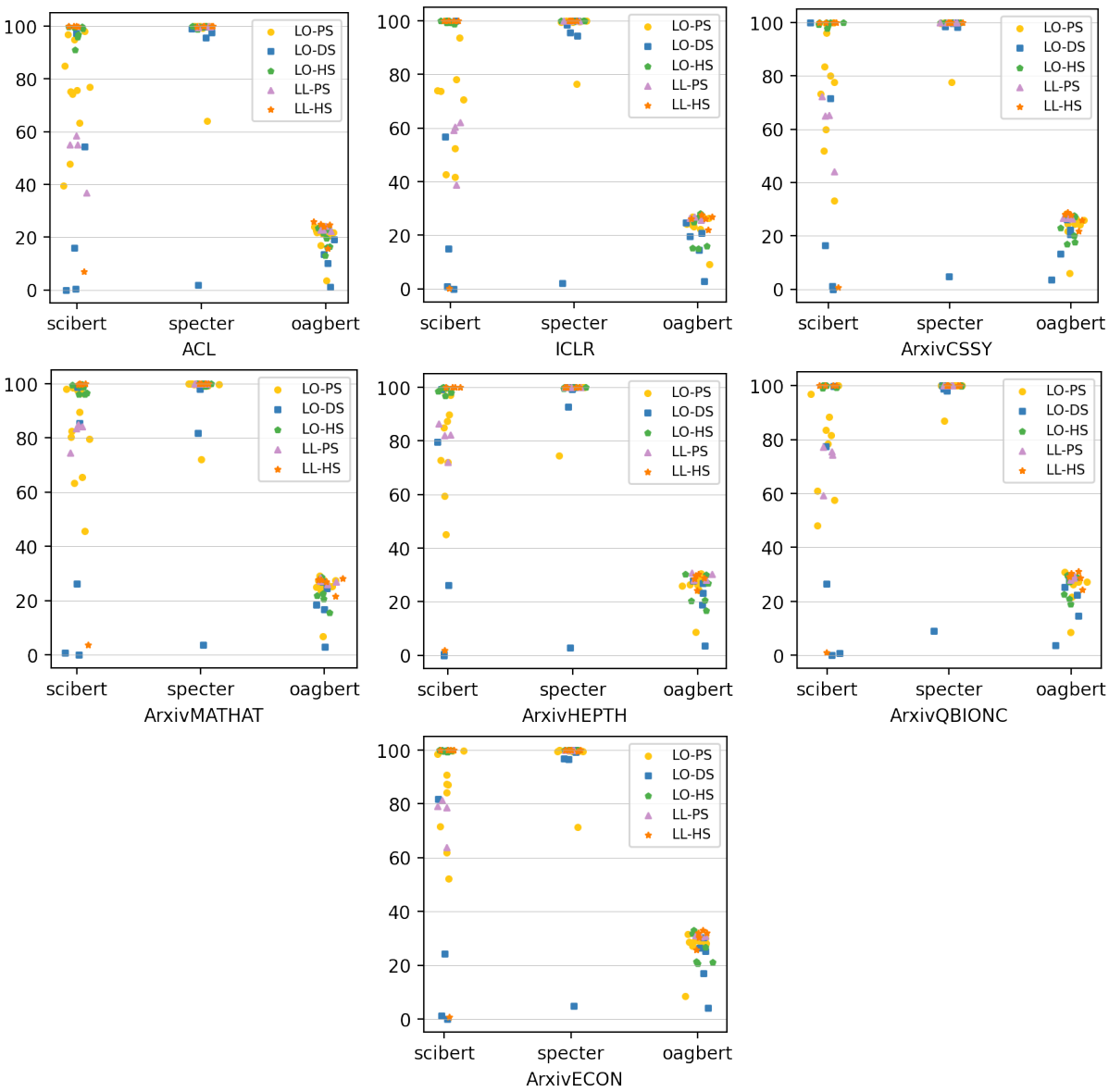


Figure 12: Distribution of NN1_Ret for each textual neighbor category. It can be observed that SciBERT embeddings preserve the hierarchy of NN1_Ret, i.e. Partially Similar categories (LO-PS and LL-PS) have lower values than Highly Similar categories (LO-HS and LL-HS).

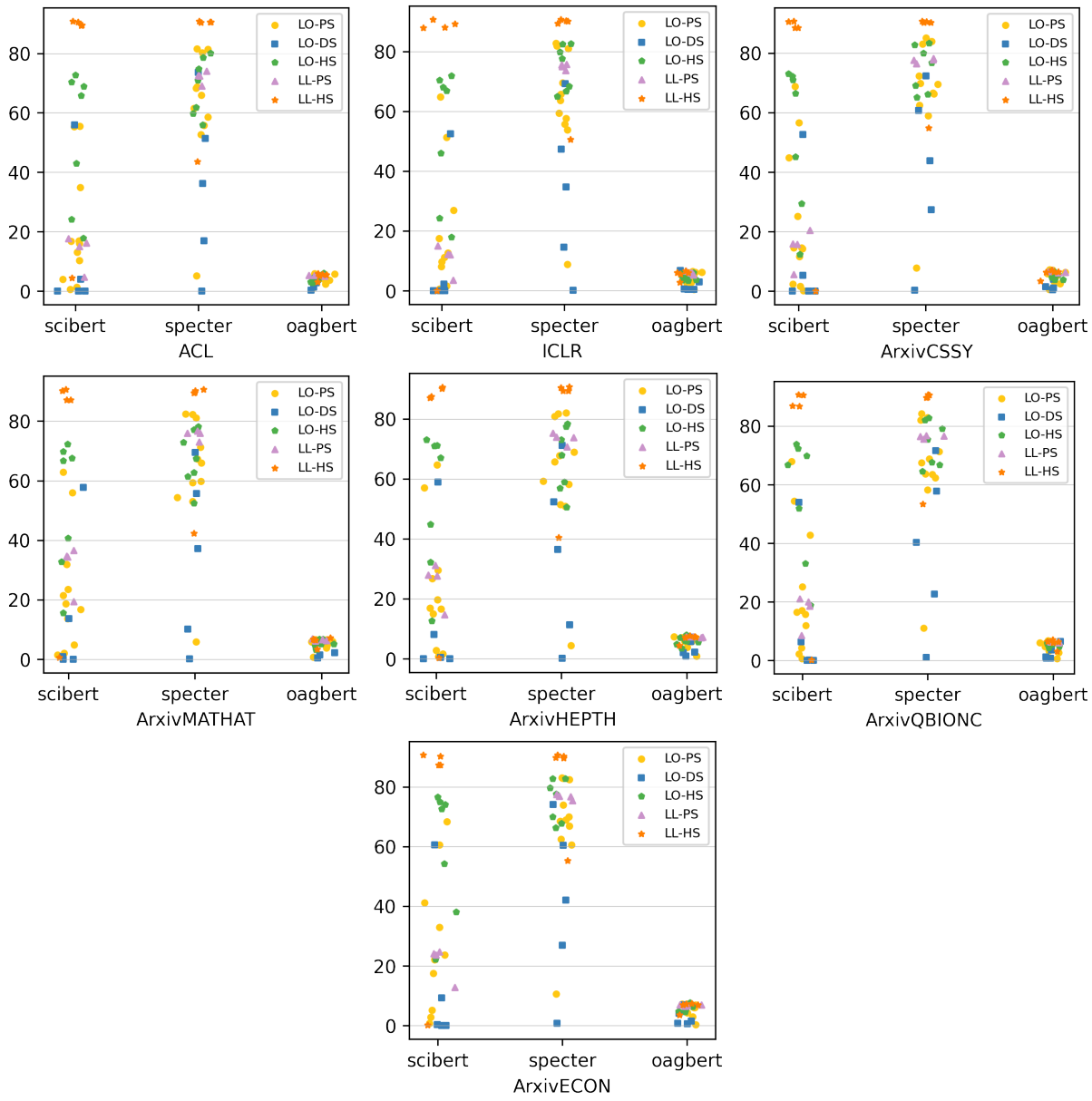


Figure 13: AOP-10 distribution of all categories of textual neighbors. It can be observed that SciBERT performs poorly for the LL-PS category (involves neighbors that scramble sentences such as arranging randomly, or arranging in increasing or decreasing order of sentence length). For the rest of the categories, SciBERT embeddings show desirable order of AOP-10 values, e.g. LL-HS $>$ LO-HS. SPECTER values have high AOP-10, which is desirable, except for the LO-DS category.